

Cryptography

Practical 2: Breaking the Vigenère Cipher

Martin Harrigan, ✉ martin.harrigan@setu.ie

1 Python Virtual Environments

We will use Python and virtual environments for our practicals to avoid dependency conflicts and ensure reproducible results. Virtual environments allow us to maintain separate Python installations with different package versions for each project, preventing one project's requirements from interfering with another's. We will use Python's built-in `venv` module to create and manage these environments, making a separate one for each practical.

Before starting, make sure you are using Python 3 by running `python --version`. To create a virtual environment, decide upon the directory where you want to place it, and run the `venv` module with the directory path:

```
$ mkdir crypto\practical-02
$ cd crypto\practical-02
$ python -m venv .venv
```

(a) Windows

```
$ mkdir -p crypto/practical-02
$ cd crypto/practical-02
$ python -m venv .venv
```

(b) Linux and macOS

Listing 1: Creating a Virtual Environment

This will create a `.venv` directory in the newly created directory. It will also create directories inside `.venv` containing a copy of the Python interpreter and various supporting files. Once you have created a virtual environment, you can activate it:

```
$ .venv\Scripts\activate.bat
```

(a) Windows

```
$ source .venv/bin/activate
```

(b) Linux and macOS

Listing 2: Activating a Virtual Environment

Activating the virtual environment will change your prompt to show the current virtual environment. You can deactivate a virtual environment with the `deactivate` command.

Within an active virtual environment, you can update `pip` and install a Python package:

```
$ pip install --upgrade pip
$ pip install art
$ python -m art text Hello
```

Listing 3: Installing a Python Package

A common convention is to save the list of installed packages to a file named `requirements.txt`. This can then be committed to version control and shipped as part of your application. You can create `requirements.txt` manually, or by using:

```
$ pip freeze > requirements.txt
```

Listing 4: Storing a List of Installed Packages

To quickly install all packages that are listed in `requirements.txt`:

```
$ pip install -r requirements.txt
```

Listing 5: Installing a List of Packages

1.1 A Python Package for the Vigenère Cipher

Add the `vigenerere` ( <https://pypi.org/project/vigenerere/>) package to your project. Use the package to encrypt the following text with the keyword `banana`:

Any person can invent a security system so clever that she or he can't think of how to break it.¹

A sample Python program is provided on the package's web page. Note that this package supports Unicode characters and not just the limited alphabet we used in the lectures (A–Z with no punctuation). Verify that you can decrypt the resulting ciphertext to recover the original plaintext.

2 Breaking the Vigenère Cipher

The ciphertext below was generated using the Vigenère Cipher using the same alphabet (A–Z with no punctuation) as in the lecture notes. Can you break it?

NTIO MLME NSML TFFW GWXI FKUK YHQB IJXT ATNA REEW NOFS FVYS LEAT QAXD AACL BAEH BWFG QCZX
TOXW MZSE EGCL BAEH BWJP AQBQ KBQZ CWKI FKUK YHQS JGHH ATCF HRQR ODNT KWNQ FSFV YKJA ECHG
KLUU BLNT IOML MEES UKTN ATXS WKZS MKNT IOML MEED LASG ATBG UEUH QSXT TSQA STQF IXIE EDUA
WWQV UVJV QFSL MIZU VWKO DSOK BETO XFTT TWHY GERC LWZS ISQW WEMZ FYTI ZUXA WEOH NGME MJYF
BEIS LWFL XUIA SGPW LWHT FVYG YHQB QSDI ZGBG WTFV YHJR UCXO FSEC ZSWL UYYL MEBF YKJN FDYJ
NOPH BSYS AAYG KIFG HGNS USML FUFV IJNT USMA SSUG NWIO ZWNK GEUB AJJC QWPW IFAF AGTD AFZG
WEHW FAST TSMM UEDZ ULNV QRYR WEQC ZUTM BOLA XOZC HDDT TSLW BEDS UCNQ SKCL MAXO LYJJ MKUF
IACI YWSW UHBS ULMW HXFC QCHL MEFV LGSE ATYF LLMB XLME DSQW WEMY CFLW UHBS QADU YBFW MBXS
VUQS HONT TOZS NRRO WWTN FVYL MRAB YGKF DOHU JIZP ILMC AIHL WIQG CLBA EQFW FRQF NZFN OFSK
YAXH ILME XCLV XORH BWXT MHYH WEES LNJS ATFG FVQG UFIF UGBW XTTO NLMI ZUMA SGQB YJFL ISLW
XEFH FWIF AFYN JRUH QSXT TSSW FRAT IMWL AFXG SEFV IMXA ZRMW AEZV OFIR QRUF ISQJ YFYR RWPW
XPUF CLZA XFYN JLMH CGSS ISLW HOZQ YVJD FCYF LLMB XSYT TONX FVAI LWIP QFCG IAEQ NLMI EALK
XOGH BUTT FVUV WEOS HLQY MHNS NNQR BWWF UJYS SDFK YFYI QHBT QEEG YVGI DHBV FYAT QZTM MDLG
UHQH CUUR UJUL JIZH BWQI RSAM FRPG BSIH QFUD IEPH BWXU NZCE JABD YSWA ZQYT DAZB IMSC UBAL
MAFO LJFN SSGW STEK YJJM MRYX TRFV YKBA XZIO NNSI JGKL ABXG SAZR QWXT YWHK YEDS PWST TSWG
HKXO HWLH AGNZ FDNS YFQA URIF QYMF IMSD PCTW SORM YSWS MTNW WRMD JASG AINA YSYS MKFG QGUK
YHQB JAWI FGIX YHUG PWWY KSUJ QAEH JSXT EIJB WNMH OJFL XMXW KIOU YFYI ZCLA LIZO FAYY DOJH
JDAI NLME UFME JRQA YKXA SSMA STTS YSWT TZSG WDQF IXJV QBNK MAPZ ULJL KQIE JTAH BWJN SZCK
MCDQ QFFN PDYG ULQT LGRA OCHY WEEG IXGR UHCK MSGP DWHT EWHB REDW WSBH UQBK YRMB AWYO DSFS
YETO PWUR AJYV RODS CEUO DHUF YTAH BWMU YOHJ FCQH BSSA ZMWG RMGB CUFT UCHK DEFF YUJI HSLX
MRAI AZFN KCZL MEOV CUPE ZGIX YHQB IUPL MBYT WOAR

¹This is known as Schneier's Law:  https://www.schneier.com/blog/archives/2011/04/schneiers_law.html.

3 How to Do It

You should spend some time trying to break the above cipher by yourself. Refer to Lecture 4: How to Break Transposition & Substitution Ciphers, in particular, the section on the Kasiski Method. In this practical we show that the weakness of the Vigenère Cipher lies in repetitions in the ciphertext, and this can be exploited to recover the original plaintext and key. Can you identify any repetitions in the above ciphertext?

3.1 Finding Repetitions

We can use the Python code in Section A to search for repetitions.

Create a text file in the same directory as the code, named `ciphertext.txt`. This file will contain the ciphertext that we want to inspect for repetitions. Place the ciphertext above into the text file. The ciphertext is also available on Blackboard as `ciphertext.txt`. You should remove all whitespace characters (spaces and newlines) so that the file contains one single line.

Run the Python program and inspect the output in the console.

```
9: {'NTIOMLMEE': 2}
8: {'NTIOMLME': 3, 'IFKUKYHQ': 2, 'WNOFSFVY': 2, 'CLBAEHBW': 2, 'TIOMLMEE': 2, ...}
7: {'NTIOMLM': 3, 'TIOMLME': 3, 'IFKUKYH': 2, 'FKUKYHQ': 2, 'WNOFSFV': 2, ...}
6: {'NTIOML': 3, 'TIOMLM': 3, 'IOMLME': 3, 'IFKUKY': 2, 'FKUKYH': 2, 'KUKYHQ': 2, ...}
5: {'NTIOM': 3, 'TIOML': 3, 'IOMLM': 3, 'OMLME': 3, 'IFKUK': 2, 'FKUKY': 2, ...}
...
NTIOMLME positions (one-based): [1, 151, 175]
Pairwise distances: [150, 174, 24]
```

This shows the repetitions in the text file. The first line indicates that the nine-character string `NTIOMLMEE` was found twice in the text file; the second line indicates that the eight-character string `NTIOMLME` was found three times in the text file, the eight-character string `IFKUKYHQ` was found twice in the text file, and the eight-character string `WNOFSFVY` was found twice in the text file, etc.

Suppose we take the entry `'NTIOMLME': 3` as a starting point. This indicates that the string `NTIOMLME` occurs three times in the ciphertext. The program prints the locations of this string, and the distances between them, on the last two lines of its output. The string `NTIOMLME` is found at the 1st, 151st and 175th character locations.

Recall from the lecture the observation of Charles Babbage that repetitions in the ciphertext suggest repetitions in the plaintext and that the distances between such repetitions are multiples of the length of the keyword. We have identified an eight-character string that repeats three times in the ciphertext. The distances between the repetitions are 150 ($151 - 1$), 174 ($175 - 1$), and 24 ($175 - 151$). According to Babbage, this suggests that 24, 150 and 174 are multiples of the length of the keyword.

The greatest common divisor (GCD) of the three numbers, 24, 150 and 174, is six. Therefore, the length of the keyword is either six, three, two or one. When choosing keywords, it is good practice to choose long keywords, therefore we will assume that the length of the keyword is six. If we are incorrect, we can revisit the other possibilities later on.

3.2 Splitting the Ciphertext

Next, we will use our assumption, that the length of the keyword is six, to try and recover the plaintext. We can use the Python code in Section B to split the ciphertext into rows where each row was encrypted using the same character from the keyword.

We need to modify the program so that it will use our keyword length of six. Find the following line in `every_nth_character.py`:

```
shifts = [0, 0, 0]
```

The number of entries in the `shifts` array should match the length of the keyword. For now, we will change it to:

```
shifts = [0, 0, 0, 0, 0, 0]
```

The `shifts` array has six entries since we believe the length of the keyword to be six. This is our way of telling the Python program that the keyword has length six, but we have no idea what the keyword is. Run the Python program and observe the output in the console. You should see the six lines below, followed by the reconstructed plaintext:

```
NMTXYXRFLXBFTSBJKKYHHNFJKNMTWNMSUXSIWJMKBTGZWTWMBFSHYDWJFWMJNYKNFNSIGJITWSUNWTX...
TEFIHTESEDAGOEAPBIHRTSALTENKTEGETTEWVIOETESIIEELGTHITRSLNOSISUTSOECFDETEVEMO...
INFFQAEFAAEQXEEAQFQAQKFEUIEAZIEAUTQEQQZDTRIMZOMIXPFQZFUEUBFPAFUFUZZUQAHTDQQBZ...
OSWKKTWVTAHCWGHQZKSTRWVCUOSTSODTHSFDVFUSOWCSZUHJSUWVFGVCCYFDHAGSVSGWBWFFWSZRCOC...
MMGUINNYQCBZMCBUCJCONYHBMUXMMLBQQIUUSVOXHLQFXNYLILYQBYXZYYYBYHMIMNNAZFMUYZLH...
LLWKJAOSALWXZLWGWKGFDOGLLKSCLAGSAXAVLWKFYWWYAGFWAGSGHOSLKJSGGLJAWKJWGGAMLYUAD...
```

The program splits the ciphertext into six lines: the first line contains every sixth character starting from the first, the second line contains every sixth character starting from the second, etc. If our keyword does indeed have six characters, then every character on the first line would have been encrypted using the first character of the keyword, every character on the second line would have been encrypted using the second character of the keyword, etc.

Each line is vulnerable to frequency analysis. If every character on the first line was encrypted using the first character of the keyword, then every character on the first line was encrypted using the same character. This is equivalent to a shift cipher or Caesar cipher. By splitting the ciphertext into six lines, we have decomposed the Vigenère cipher into six separate Caesar ciphers. The question remains, can we decipher each line?

We return to the `shifts` array in `every_nth_character.py`. The six entries correspond to the six separate Caesar ciphers. The value for each entry is the key for each Caesar cipher. Suppose we change the first value to 1:

```
shifts = [1, 0, 0, 0, 0, 0]
```

If we re-run the Python program, we observe:

```
MLSWXWQEKWAESRAIJJXGMEIJMLSVMLRTWRHVILJASFYVSVLAERGXCIVIEVIMXJMEMRHFHSVRTMVSW...
TEFIHTESEDAGOEAPBIHRTSALTENKTEGETTEWVIOETESIIEELGTHITRSLNOSISUTSOECFDETEVEMO...
INFFQAEFAAEQXEEAQFQAQKFEUIEAZIEAUTQEQQZDTRIMZOMIXPFQZFUEUBFPAFUFUZZUQAHTDQQBZ...
OSWKKTWVTAHCWGHQZKSTRWVCUOSTSODTHSFDVFUSOWCSZUHJSUWVFGVCCYFDHAGSVSGWBWFFWSZRCOC...
MMGUINNYQCBZMCBUCJCONYHBMUXMMLBQQIUUSVOXHLQFXNYLILYQBYXZYYYBYHMIMNNAZFMUYZLH...
LLWKJAOSALWXZLWGWKGFDOGLLKSCLAGSAXAVLWKFYWWYAGFWAGSGHOSLKJSGGLJAWKJWGGAMLYUAD...
```

Note that only the first line has changed. Does this line look like it contains every sixth character from a piece of English text? No, it does not. There are too few instances of T and too many instances of V and X. We try the next value:

```
shifts = [2, 0, 0, 0, 0, 0]
```

We re-run the Python program and inspect the output. We repeat this process until the first line looks like it does contain every sixth character from a piece of English text. This occurs with the value 5:

```
shifts = [5, 0, 0, 0, 0, 0]
```

The corresponding output is:

```
IHOSTMAGSWAONWEFFTCCIAEFIHORIHNPSNDREHFWOBURORHWANCTYREARHEITFIAINDBEDORNPIROS...
TEFIHTESEDAGOEAPBIHRTSALTENKTEGETTEWVIOETESIIEELGTHITRSLNOSISUTSOECFDETEVEMO...
INFFQAEFAAEQXEEAQFQAQKFEUIEAZIEAUTQEQQZDTRIMZOMIXPFQZFUEUBFPAFUFUZZUQAHTDQQBZ...
OSWKKTWVTAHCWGHQZKSTRWVCUOSTSODTHSFDVFUSOWCSZUHJSUWVFGVCCYFDHAGSVSGWBWFFWSZRCOC...
MMGUINNYQCBZMCBUCJCONYHBMUXMMLBQQIUUSVOXHLQFXNYLILYQBYXZYYYBYHMIMNNAZFMUYZLH...
LLWKJAOSALWXZLWGWKGFDOGLLKSCLAGSAXAVLWKFYWWYAGFWAGSGHOSLKJSGGLJAWKJWGGAMLYUAD...
```

The value 5 indicates that the first letter of the keyword is F, since F is five places to the right of A. We repeat this process for the next line. In fact, the next line already looks like it could contain every sixth character from a piece of English text. There are lots of Es and Ts. Therefore, we will leave the second value at 0. This value indicates that the second letter of the keyword is A, since A is zero places to the right of A.

We repeat the process for the third line. This requires checking several values. Eventually, you should suspect the value 12. We repeat for the remaining lines. In the end, you should have:

```
shifts = [5, 0, 12, 14, 20, 18]
```

What is the plaintext? What is the keyword?

References

- [1] P. Cochrane. Basic Python Project Setup.  <https://peateasea.de/basic-python-project-setup/>, 2024.

Appendices

A Python Code for Finding Repetitions in a String

```
import re

# Kasiski examination. Finds repeated substrings in ciphertext to help
# determine the key length of a polyalphabetic cipher (e.g., Vigenère).

INPUT_PATH = "ciphertext.txt"

WHITESPACE_PATTERN = re.compile(r"\s+")

def read_input(path):
    """Reads the contents of a file as a string, removes all whitespace
    and returns the contents as a string."""
    with open(path) as file:
        string = file.read().rstrip()
    return re.sub(WHITESPACE_PATTERN, "", string)

def find_repetitions(string, slen):
    """Count repeated substrings, including overlapping occurrences."""
    return {text: len(positions) for text, positions in find_positions(string, slen).items()}

def find_positions(string, slen):
    """Return repeated substrings and their zero-based starting positions."""
    if slen < 1:
        raise ValueError("Substring length must be positive")
    positions = {}
    for i in range(len(string) - slen + 1):
        subs = string[i : i + slen]
        positions.setdefault(subs, []).append(i)
    return {text: starts for text, starts in positions.items() if len(starts) > 1}

if __name__ == "__main__":
    input_ = read_input(INPUT_PATH)
    # Bound the search; extend this range if longer repeats are of interest.
    for length in range(min(12, len(input_)), 2, -1):
        counts = find_repetitions(input_, length)
        if counts:
            print(f"{length}: {counts}")
    starts = find_positions(input_, 8).get("NTIOMLME", [])
    print("NTIOMLME positions (one-based):", [i + 1 for i in starts])
    print("Pairwise distances:", [b - a for i, a in enumerate(starts) for b in starts[i + 1 :]])
```

Listing 6: repetition_finder.py

B Python Code for Extracting Every *i*th Character from a String

```
import re

# Vigenère cipher analysis. Extracts every i-th character to isolate
# each key position, then applies a Caesar shift to decrypt.

INPUT_PATH = "ciphertext.txt"

WHITESPACE_PATTERN = re.compile(r"\s+")

def read_input(path):
    """Reads the contents of a file as a string, removes all whitespace
    and returns the contents as a string."""
    with open(path) as file:
        string = file.read().rstrip()
    return re.sub(WHITESPACE_PATTERN, "", string)

def unshift_char(char, unshift):
    return chr((ord(char) - ord("A") - unshift) % 26 + ord("A"))

def decrypt(string, shifts):
    if not shifts or any(char < "A" or char > "Z" for char in string):
        raise ValueError("Use a nonempty key and uppercase A-Z ciphertext")
    return "".join(unshift_char(char, shifts[i % len(shifts)]) for i, char in enumerate(string))

if __name__ == "__main__":
    input_ = read_input(INPUT_PATH)
    shifts = [0, 0, 0]
    plaintext = decrypt(input_, shifts)
    for i in range(len(shifts)):
        print(plaintext[i :: len(shifts)])
    print("Reconstructed plaintext:")
    print(plaintext)
```

Listing 7: every_ith_character.py